

SFML nel progetto Lotka-Volterra

Spiegazione dettagliata di PlotAxes, Plotter, OrbitPlotter e dell'uso della libreria SFML — preparazione orale di programmazione

1. Concetti SFML di base

`sf::RenderWindow` e `sf::RenderTarget`

`sf::RenderWindow` è la finestra grafica vera e propria: creandola con `sf::RenderWindow(sf::VideoMode(w, h), "titolo")` si apre una finestra $w \times h$ pixel. È anche una superficie su cui disegnare, perché **eredita** da `sf::RenderTarget`, una classe base astratta che rappresenta "qualcosa su cui posso chiamare `.draw(...)`". Per questo tutte le funzioni di disegno (`drawAxes`, `drawPlot`, `drawOrbit`) prendono un parametro `sf::RenderTarget& target` invece di `sf::RenderWindow&`: così la stessa funzione disegna indifferentemente sulla finestra principale o su quella dell'orbita, senza doverla duplicare per ognuna — puro polimorfismo.

Il sistema di coordinate

In SFML l'origine (0,0) è in **alto a sinistra**, e l'asse Y cresce **verso il basso** (al contrario della convenzione matematica). Da qui il `1.0` - presente in ogni trasformazione coordinate→pixel: senza, i grafici verrebbero disegnati capovolti, con i valori alti in basso.

`sf::VideoMode` e ciclo di vita della finestra

`sf::VideoMode` incapsula una risoluzione (larghezza, altezza, opzionalmente bit per pixel); serve solo a descrivere "che dimensioni deve avere la finestra", non ha altri usi nel nostro codice dato che non lavoriamo in fullscreen. Il costruttore di `sf::RenderWindow`, se ricevuto un `sf::VideoMode` e un titolo, **apre già la finestra sullo schermo**: non esiste un metodo separato "create" da chiamare dopo, come invece accade in altre librerie grafiche. Questo significa anche che il momento in cui l'oggetto `sf::RenderWindow` viene costruito è il momento in cui il sistema operativo alloca risorse reali (finestra, contesto grafico): per questo, quando serve creare una finestra su un thread diverso da quello principale (come in `OrbitPlotter`), la costruzione dell'oggetto deve avvenire **dentro** quel thread, non prima.

`sf::Event`: non solo `Closed`

`sf::Event` è una union/struct che rappresenta un singolo evento di input o di sistema: chiusura finestra, tasto premuto, movimento del mouse, ridimensionamento, perdita del focus, e così via. Il campo `event.type` dice quale evento sia effettivamente successo, e in base a quello si può accedere ai campi specifici della union (es. `event.key.code` per un tasto, `event.mouseButton.x/y` per un click). Nel nostro codice gestiamo solo `sf::Event::Closed` (l'utente ha premuto la X della finestra), ignorando tutto il resto: gli altri eventi vengono comunque "consumati" dal ciclo `pollEvent` (altrimenti si accumulerebbero in coda all'infinito), semplicemente non fanno scattare nessuna azione.

Il ciclo principale (game loop)

Sia `Plotter::show()` sia `OrbitPlotter::show()` seguono lo stesso schema:

```
while (window.isOpen()) {
    sf::Event event;
    while (window.pollEvent(event)) {          // 1. svuota la coda eventi
        if (event.type == sf::Event::Closed)
            window.close();
    }
    window.clear(sf::Color(245,245,245));      // 2. pulisci il frame precedente
    // ... disegna ...                        // 3. disegna tutto da zero
    window.display();                          // 4. mostra il frame
}
```

`pollEvent` è non bloccante: svuota la coda degli eventi accumulati (può essercene più di uno per frame), e ritorna false quando non ce ne sono più — per questo è dentro un while interno. `setFramerateLimit(60)` dice a SFML di non disegnare più di 60 frame al secondo, per non sprecare CPU ridisegnando migliaia di volte al secondo un'immagine statica.

RAII: perché le finestre SFML non si chiudono mai esplicitamente

In tutto il progetto non si trova mai una chiamata esplicita tipo `window.destroy()` o simili per liberare le risorse della finestra: quando `window_` (in `Plotter`) o `window` (variabile locale in `OrbitPlotter::show`) escono dallo scope, il loro **distruttore** viene chiamato automaticamente e si occupa di rilasciare tutte le risorse SFML/OpenGL sottostanti. Questo è il principio **RAII** (Resource Acquisition Is Initialization), pilastro del C++ moderno: una risorsa (qui, la finestra col suo contesto grafico) viene acquisita nel costruttore e rilasciata automaticamente nel distruttore, legando il suo ciclo di vita a quello dell'oggetto C++ che la rappresenta. Lo stesso vale per `sf::Font`: il font caricato in memoria da `loadFromFile` viene liberato automaticamente quando l'oggetto `sf::Font` esce dallo scope, senza bisogno di una chiamata tipo `unloadFont()`.

Perché si ridisegna tutto ad ogni frame

Un dubbio frequente: perché `clear()` e poi ridisegnare tutto da capo ogni volta, se il grafico è statico? La risposta è il **double buffering**: SFML (come quasi tutte le librerie basate su OpenGL) mantiene due buffer immagine, uno mostrato a schermo e uno su cui si sta disegnando; `window.display()` scambia i due. Se non si ridisegnasse tutto ad ogni frame, al primo scambio si vedrebbe un frame vuoto o con contenuto vecchio. Per un contenuto statico come il nostro questo è "spreco" trascurabile, ulteriormente limitato da `setFramerateLimit(60)`: ridisegnare poche linee e rettangoli 60 volte al secondo costa pochissimo a CPU/GPU.

2. PlotAxes.hpp / PlotAxes.cpp

Contiene due funzioni **libere** (non membri di classe), nel namespace `lotka_volterra`.

`loadAnyFont(sf::Font& font)`

sf::Font rappresenta un font caricato in memoria; serve sempre per disegnare testo (`sf::Text`). SFML **non include un font di default**: va caricato da un file `.ttf` con `font.loadFromFile(percorso)`, e il percorso dipende dal sistema operativo. `kFontPaths` elenca percorsi tipici (Linux, macOS, Windows); la funzione li prova uno per uno finché uno funziona. `loadFromFile` ritorna `bool` invece di lanciare eccezioni (stile tipico di SFML). Se nessun font viene trovato la funzione ritorna `false`: il programma non crasha, semplicemente non disegnerà le etichette di testo (per questo in giro nel codice si trova sempre `if (fontLoaded) { ... }`).

`drawAxes(...)`

sf::FloatRect: un rettangolo (`left`, `top`, `width`, `height`) che rappresenta il riquadro dentro cui disegnare un grafico.

sf::RectangleShape disegna il bordo del grafico:

```
sf::RectangleShape border({area.width, area.height});
border.setPosition(area.left, area.top);
border.setFillColor(sf::Color::White);
border.setOutlineColor(sf::Color(60, 60, 60));
border.setOutlineThickness(1.f);
target.draw(border);
```

sf::Color si costruisce con costanti (`sf::Color::White`, `::Blue...`) oppure con `sf::Color(r,g,b)` (0-255) per colori custom. Per le righe della griglia si usa un approccio più a basso livello: un array di 2 `sf::Vertex` (posizione + colore) disegnato come `sf::Lines`, cioè interpretando i vertici a **coppie**, ciascuna un segmento indipendente. `sf::Vector2f` è semplicemente una coppia (x, y) di float, usata ovunque servano posizioni. Le etichette numeriche usano **sf::Text**: costruttore (stringa, riferimento al font, dimensione in pixel). Questa funzione è interamente stateless (non tocca membri di classe: riceve font e `fontLoaded` come parametri), per questo può essere condivisa da `Plotter` e `OrbitPlotter` senza che l'una dipenda dall'altra.

Perché le etichette sono opzionali (if fontLoaded)

Si noti che gli assi, la griglia e le linee dei grafici vengono sempre disegnati, indipendentemente dal fatto che il font sia stato caricato o meno; solo le chiamate a `sf::Text` sono racchiuse in `if (fontLoaded) { ... }`. Questa è una scelta di robustezza: se il programma girasse su una macchina priva di uno qualunque dei font in `kFontPaths` (ad esempio un ambiente Linux minimale), il grafico sarebbe comunque perfettamente leggibile nella sua struttura (assi, griglia, curve, colori), semplicemente senza numeri sugli assi e senza titoli testuali — molto meglio che far crashare il programma con un'eccezione o una asserzione fallita per un font mancante, che è un problema di deployment e non di logica del programma.

Un dettaglio sulla normalizzazione: divisione per zero

La formula $(value - min) / (max - min)$, usata sia per gli assi sia per le curve, avrebbe un problema se `max` fosse uguale a `min` (ad esempio una serie costante, o un solo campione): si avrebbe una divisione per zero. Sia `drawPlot` sia `drawOrbit` se ne proteggono con un controllo esplicito prima di chiamare `drawAxes`: `if (vMax - vMin < 1e-9) vMax = vMin + 1.0;` forza un intervallo minimo non nullo. Il confronto con `1e-9` anziché con `0` esatto è una precauzione tipica quando si lavora con `double`: per via degli errori di arrotondamento in virgola mobile, due valori teoricamente uguali potrebbero differire di una quantità infinitesima ma non esattamente zero.

3. Plotter.hpp / Plotter.cpp — grafico nel tempo

Il costruttore

```
Plotter::Plotter(unsigned int width, unsigned int height, const std::string& title)
    : window_(sf::VideoMode(width, height), title), fontLoaded_(false)
{
    window_.setFramerateLimit(60);
    fontLoaded_ = loadAnyFont(font_);
}
```

sf::VideoMode(width, height) descrive una risoluzione; qui serve solo a dire alla finestra le dimensioni da avere. `window_` viene costruita già nella lista di inizializzazione: le finestre SFML si aprono alla costruzione dell'oggetto `sf::RenderWindow`.

drawPlot() — trasformazione dati → pixel

```
sf::VertexArray line(sf::LineStrip, time.size());
for (std::size_t i = 0; i < time.size(); ++i) {
    const float fx = area.left + area.width *
        static_cast<float>((time[i] - tMin) / (tMax - tMin));
    const float fy = area.top + area.height *
        static_cast<float>(1.0 - (s.values[i] - vMin) / (vMax - vMin));
    line[i] = sf::Vertex(sf::Vector2f(fx, fy), s.color);
}
target.draw(line);
```

sf::VertexArray è come un array di **sf::Vertex** ma dinamico (dimensione decisa a runtime). Il primo argomento del costruttore, il **sf::PrimitiveType**, dice come interpretare la sequenza di punti: **sf::Lines** interpreta i vertici a coppie (segmenti indipendenti), mentre **sf::LineStrip** (qui) li collega in sequenza 0→1→2→3..., formando una linea spezzata continua — perfetto per una curva $y(t)$. La formula normalizza $time[i]$ tra $tMin$ e $tMax$ in un valore 0-1, poi lo scala su $area.width$ e lo trasla di $area.left$ per ottenere il pixel fx ; stessa cosa per fy , con l'inversione dell'asse Y. Ogni vertice riceve il colore della serie ($s.color$), quindi l'intera linea viene disegnata di quel colore.

drawLegend() e show()

drawLegend disegna, per ogni serie, un rettangolino colorato (**sf::RectangleShape** 14×4 px) seguito dal nome (**sf::Text**), spostando y di 20px ad ogni iterazione: puro posizionamento manuale, nessuna magia SFML. **show()** è il game loop descritto in apertura; chiama **drawPlot** due volte per frame — una per **topArea** (popolazioni) e una per **bottomArea** ($H(t)$) — per questo in **main.cpp** si passa **hSeries** avvolta in **{hSeries}**, per farla diventare un vector di un elemento (**drawPlot** si aspetta sempre un **vector<Series>**).

Il titolo in grassetto: sf::Text::setStyle

Nel titolo di ogni riquadro ("Popolazioni nel tempo", "Integrale primo $H(t)$ ") si vede `titleText.setStyle(sf::Text::Bold)`. **sf::Text::Bold** è una delle costanti dell'enum **sf::Text::Style** (insieme a **Italic**, **Underlined**, **StrikeThrough**, **Regular**); a differenza di **setFillColor** o **setPosition**, che agiscono su proprietà geometriche/cromatiche, **setStyle** modifica il modo in cui i glifi del font vengono renderizzati (SFML applica una trasformazione allo stroke dei caratteri per simulare il grassetto, dato che il font caricato è uno solo, non una famiglia **regular/bold** separata). Gli stili si possono anche combinare con l'operatore bit a bit OR, ad esempio `sf::Text::Bold | sf::Text::Italic`.

Perché window_, font_ e fontLoaded_ sono privati

I tre membri di **Plotter** (**sf::RenderWindow window_**, **sf::Font font_**, **bool fontLoaded_**) sono dichiarati **private** nell'header: lo stato interno della finestra non deve poter essere manipolato dall'esterno della classe, solo attraverso l'interfaccia pubblica **show()**. Questo è incapsulamento standard, ma qui ha anche un significato pratico legato a SFML: se qualcuno esternamente sostituisse **window_** o lo richiudesse a metà di un ciclo di disegno, si romperebbe l'invariante "finché **show()** gira, **window_** è la stessa finestra aperta all'inizio", causando comportamento indefinito nelle chiamate a **draw()**.

4. OrbitPlotter.hpp / OrbitPlotter.cpp — orbita nel piano

Perché è tutto static

Qui non c'è nessun membro **window_/font_** di classe: finestra e font vengono creati **dentro** **show()**, come variabili locali. Il motivo è che questa finestra deve poter girare **su un thread separato** rispetto a quella principale (**Plotter**). Ogni **sf::RenderWindow** porta con sé un **contesto OpenGL**, e SFML/OpenGL non gestiscono bene un contesto attivato e disattivato avanti e indietro tra thread diversi. La soluzione più robusta è: ogni finestra vive interamente sul thread che l'ha creata, dalla creazione alla chiusura, senza condividere nulla con l'altra. Per questo **window** e **font** sono locali a **show()** invece che membri persistenti, e **show()** è static: non serve istanziare la classe perché non c'è nessuno stato da mantenere tra una chiamata e l'altra.

drawMarker — una lambda con sf::CircleShape

```
auto drawMarker = [&](double xv, double yv, sf::Color color) {
    const float fx = ...; const float fy = ...;
    sf::CircleShape marker(4.f);
    marker.setFillColor(color);
    marker.setOrigin(4.f, 4.f);
    marker.setPosition(fx, fy);
    target.draw(marker);
};
drawMarker(x_values.front(), y_values.front(), sf::Color::Green);
drawMarker(x_values.back(), y_values.back(), sf::Color::Red);
```

sf::CircleShape(raggio) è un cerchio disegnabile. Il dettaglio SFML importante è **setOrigin(4.f, 4.f)**: di norma **setPosition** posiziona l'angolo in alto a sinistra della bounding box della forma. Per un cerchio di raggio 4, quell'angolo è (centro.x-4, centro.y-4), non il centro. **setOrigin** sposta il punto di riferimento della forma al suo centro (4,4), così

setPosition(fx, fy) posiziona esattamente il **centro** del cerchio nel punto che si vuole marcare — senza, il pallino sarebbe visibilmente spostato in basso a destra. La lambda [&] cattura per riferimento tutto ciò che le serve (target, area, xMin...xMax) per non ripetere la formula di trasformazione due volte (marker verde di partenza, rosso di arrivo).

drawOrbit() vs drawPlot()

Concettualmente identica a drawPlot, ma qui non c'è un asse tempo: l'asse orizzontale è x_values (prede), quello verticale y_values (predatori) — è un grafico nello spazio delle fasi. Per questo xMin/xMax/yMin/yMax si calcolano con std::min_element/std::max_element (STL, non SFML) sui due vettori, invece di usare front()/back() come per il tempo (che è ordinato crescente per costruzione; x e y invece oscillano).

I marker verde/rosso come convenzione visiva

sf::Color::Green e sf::Color::Red per i due marker non sono una scelta arbitraria di palette, ma una convenzione comune nelle visualizzazioni di traiettorie: verde = inizio, rosso = fine, la stessa logica di un semaforo o di molte interfacce di editing video. Serve in particolare a leggere a colpo d'occhio se l'orbita si è richiusa su sé stessa (come previsto dalla teoria per un sistema Lotka-Volterra, che conserva H): se il marker rosso finisce visivamente sovrapposto (o molto vicino) a quello verde, la simulazione ha compiuto un periodo completo senza derive numeriche apprezzabili.

Perché drawOrbit è privata e show è pubblica

drawOrbit() è dichiarata private nell'header: rappresenta un dettaglio implementativo di **come** disegnare l'orbita dentro un'area già data, non fa parte del contratto che OrbitPlotter offre all'esterno. L'unica cosa che il resto del programma deve poter fare è "apri una finestra e mostrami l'orbita", cioè show(): questo è lo stesso principio di incapsulamento visto in Plotter, applicato qui a una classe con soli membri statici. Separare interfaccia pubblica minimale e dettagli privati rende più facile, in futuro, cambiare il modo in cui l'orbita viene disegnata (ad esempio aggiungendo una griglia polare) senza toccare il codice che la usa.

5. main.cpp — il vero multithreading

```
std::thread orbitThread(&OrbitPlotter::show, std::cref(x_values), std::cref(y_values));
```

```
Plotter plotter;  
plotter.show(time, populationSeries, hSeries);
```

```
orbitThread.join();
```

Questo non è SFML ma <thread> della libreria standard — vale la pena capirlo bene perché è il punto in cui le due finestre prendono vita in parallelo. **std::thread(funzione, argomenti...)** crea e avvia subito un nuovo thread di sistema operativo che esegue funzione(argomenti...): qui OrbitPlotter::show, un puntatore a funzione statica (si passa con &NomeClasse::nomeMetodo perché, essendo static, non serve un'istanza). **std::cref(x_values)**: dato che show vuole i parametri per const&, e std::thread di norma **copia** tutti gli argomenti (per sicurezza, nel caso il thread chiamante finisse prima), std::cref forza invece il passaggio per riferimento costante — sicuro qui perché si aspetta la fine del thread con join() prima che main esca, quindi i vettori restano vivi per tutta la durata in cui il thread li usa. Nel frattempo il thread principale prosegue subito con plotter.show(...): da questo momento ci sono **due finestre SFML aperte contemporaneamente**, ciascuna con il proprio ciclo eventi sul proprio thread, come richiesto dal vincolo "un contesto OpenGL per thread". **orbitThread.join()** blocca main finché il thread dell'orbita non termina (l'utente chiude anche quella finestra): senza, il programma potrebbe terminare distruggendo tutto mentre l'altro thread sta ancora disegnando — comportamento indefinito.

6. Riepilogo delle classi/funzioni SFML usate

Classe / funzione	A cosa serve	Dove
sf::RenderWindow	Finestra grafica; possiede il contesto OpenGL ed espone isOpen/pollEvent/clear/display.	Plotter, OrbitPlotter
sf::RenderTarget	Interfaccia astratta per "qualcosa su cui disegnare"; RenderWindow la implementa.	PlotAxes, Plotter, OrbitPlotter
sf::VideoMode	Descrive una risoluzione (w×h) da passare al costruttore della finestra.	Plotter, OrbitPlotter
sf::Font / sf::Text	Caricamento di un font da file e disegno di stringhe di testo con quel font.	PlotAxes
sf::Color	Colore RGB; costanti predefinite o sf::Color(r,g,b) custom.	tutti i file
sf::Vector2f	Coppia (x, y) di float; usata per posizioni e dimensioni.	tutti i file
sf::FloatRect	Rettangolo (left, top, width, height); rappresenta l'area di un grafico.	tutti i file

Classe / funzione	A cosa serve	Dove
sf::RectangleShape	Forma rettangolare disegnabile (bordo del grafico, swatch della legenda).	PlotAxes, Plotter
sf::CircleShape	Forma circolare disegnabile; usata con setOrigin per i marker sull'orbita.	OrbitPlotter
sf::Vertex / sf::VertexArray	Punti con colore, disegnati come primitive (Lines, LineStrip) per griglie e curve.	PlotAxes, Plotter, OrbitPlotter
sf::Event	Evento di sistema (chiusura finestra, tastiera, mouse...) letto con pollEvent.	Plotter, OrbitPlotter

7. Domande tipiche d'orale su questo codice

Cosa succederebbe se togliessi orbitThread.join()?

main potrebbe terminare (ed eseguire i distruttori di x_values/y_values, tra le altre cose) mentre il thread dell'orbita è ancora nel suo game loop e li sta leggendo per riferimento: si tratterebbe di un comportamento indefinito (undefined behavior), non necessariamente un crash immediato ma un errore latente. Inoltre, se un std::thread viene distrutto senza essere stato né join()ato né detach()ato, il distruttore chiama std::terminate() e il programma abortisce subito.

Perché OrbitPlotter non condivide la finestra o il font con Plotter?

Perché le due finestre girano su thread diversi, e SFML richiede che un contesto OpenGL sia usato in modo coerente da un solo thread alla volta: la soluzione adottata è che ogni finestra crei e possiede tutto localmente, sul proprio thread.

Perché drawAxes e loadAnyFont sono funzioni libere e non metodi di una classe?

Non hanno bisogno di nessuno stato persistente (this): ricevono tutto l'occorrente come parametro. Renderle libere permette di condividerle tra Plotter e OrbitPlotter senza creare una relazione di ereditarietà o composizione tra le due.

Cosa cambia tra sf::Lines e sf::LineStrip?

Entrambi sono sf::PrimitiveType passati a un sf::VertexArray. Con sf::Lines i vertici sono a coppie: ogni coppia è un segmento indipendente (righe della griglia, scollegate tra loro). Con sf::LineStrip i vertici sono collegati in sequenza, formando un'unica linea spezzata continua (curve x(t), y(t), H(t) e orbita).

Perché setOrigin(4.f, 4.f) è necessario per i marker dell'orbita?

Perché setPosition posiziona di norma l'angolo in alto a sinistra della bounding box di una forma, non il suo centro. Per un cerchio di raggio 4, quell'angolo è (centro.x-4, centro.y-4). Spostando l'origine al punto (4,4) — cioè al centro del cerchio, dato il raggio 4 — setPosition(fx, fy) posiziona esattamente il centro del marker nel punto (fx, fy) calcolato dalla trasformazione dati→pixel.

Perché il costruttore di Plotter usa una lista di inizializzazione invece di assegnare window_ nel corpo?

sf::RenderWindow non ha (e non avrebbe senso che avesse) un costruttore di default utile: va costruita già con una risoluzione e un titolo, quindi deve essere inizializzata nella member initializer list. Se si provasse ad assegnarla nel corpo del costruttore, servirebbe comunque prima costruirla con un valore, il che non risolverebbe nulla e aggiungerebbe una costruzione temporanea inutile: la initializer list evita questo passaggio superfluo costruendo l'oggetto già nel suo stato finale.

join() e detach(): qual è la differenza e perché qui si usa join()?

join() blocca il thread chiamante finché il thread lanciato non ha terminato la sua esecuzione, dopodiché "raccolge" le sue risorse di sistema. detach() invece stacca subito il thread lanciato da quello std::thread, che continua a girare in background in modo completamente indipendente, senza più nessun modo per aspettarlo o sincronizzarsi con esso. Qui serve join() e non detach() per due motivi: primo, main non deve terminare (e distruggere x_values/y_values) mentre il thread dell'orbita li sta ancora leggendo; secondo, il programma deve restare in esecuzione finché l'utente non ha chiuso entrambe le finestre, non solo quella principale.

Perché sf::VertexArray e non semplicemente tante chiamate a target.draw() singole?

Per efficienza: ogni chiamata a target.draw() comporta un certo overhead (setup dello stato di rendering, eventuale cambio di shader/texture). Raggruppare tutti i punti di una curva in un solo sf::VertexArray permette a SFML di inviarli alla GPU in una singola chiamata di disegno (una singola "draw call"), molto più efficiente che disegnare centinaia di piccoli segmenti separati uno per uno, specialmente con simulazioni che generano migliaia di passi temporali.

std::cref serve solo per evitare una copia, o c'è anche un altro motivo?

Il motivo principale è di correttezza, non solo di efficienza: OrbitPlotter::show si aspetta parametri per const std::vector<double>&. std::thread però decade e copia gli argomenti per valore per costruzione, salvo quando gli si passa esplicitamente uno std::reference_wrapper (che è ciò che std::ref/std::cref restituiscono). Senza cref, il thread

lavorerebbe su una copia locale di `x_values/y_values` invece che sui vettori reali costruiti in `main` — funzionerebbe comunque (dato lo stesso contenuto), ma sprecherebbe memoria e tempo copiando vettori potenzialmente grandi (uno per ogni passo della simulazione).